

Dictionaries and sets

Introduction to Programming

Rae Harbird, r.harbird@ucl.ac.uk

Sets

Dictionaries

Questions

Sets

Sets

A set is a collection of unique items.



A set of Toby Jugs, Birmingham Museum via Wikimedia

Creating a Set

- ▶ A set is created using a set literal or the set function.

```
studentNamesSet = { "Bogdan Ionita", "Xiang Li", "Asim Ali" }
```

- ▶ You cannot use {} to make an empty set in Python.
- ▶ Instead, use the set() function with no arguments or you can use the set function to convert any sequence into a set:

```
studentNamesList = ["Bogdan Ionita", "Xiang Li", "Asim Ali"]
```

```
studentNamesSet = set(studentNamesList)
```

Set length and testing membership

- ▶ As with other python containers, you can use the `len()` function to obtain the number of elements in a set:

```
numberOfStudents = len(studentNamesSet)
```

- ▶ The `in` operator is used to test whether an element is a member of a set.

```
if "Xiang Li" in studentNamesSet :  
    print("Xiang is taking COMP0015.")  
else :  
    print("Xiang is not registered on COMP0015.")
```

- ▶ To determine whether an element is not contained in the set, use the `not in` operator.

Traversing a set

- ▶ Sets are unordered, you cannot access the elements of a set by position as you can with a list. Instead, use a `for` loop:

```
print("COMP0015 in 2019:")  
for student in studentNamesSet :  
    print(student)
```

- ▶ Note that the order of the elements in the output is different from the order in which the set was created. This is so that the elements can be stored and managed in the most efficient way possible.
- ▶ To use the elements in sorted order use the `sorted` function, which returns a list (not a set):

```
for student in sorted(studentNamesSet) :  
    print(student)
```

Adding items to a set

Just like other Python containers, you can use `add` to add items:

```
studentNamesSet.add("Ella White")
```

- ▶ **Remember** that a set cannot contain duplicate elements. If you attempt to add an element that is already in the set, there is no effect and the set is not changed.
- ▶ Take a look in the examples folder at the program `course_manager.py`. Run it and look at the code.

Removing set elements

- ▶ There are two methods that can be used to remove individual elements from a set. The `discard` method removes an element if the element exists:

```
studentNamesSet.discard("Ella White")
```

- ▶ Again, it has no effect if the given element is not a member of the set.
- ▶ The `remove` method, on the other hand, removes an element if it exists, but raises an exception if the given element is not a member of the set:

```
studentNamesSet.remove("Ella White")    # Raises an exception
```

- ▶ We have not learned about exceptions yet so we will come back to this later.

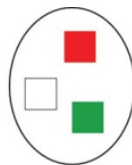
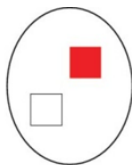
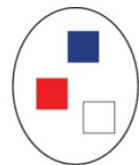
Emptying a set

- ▶ Finally, the `clear` method removes all elements of a set, leaving the empty set:

```
studentNamesSet.clear()    # set now has size 0
```

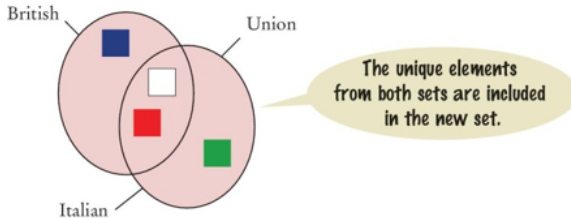
Subset, union, intersection and difference

- ▶ Set operations such as subset, union, intersection and difference can be best illustrated with an example. Imagine that a country can be represented by the set of the colours in its flag:
 - ▶ Britain is represented by the set {"red", "white", "blue"}
 - ▶ Canada is {"red", "white"}
 - ▶ Italy is {"red", "white", "green"}.



Union

- ▶ The `union` of two sets is the elements that appear in both sets without including duplicates.



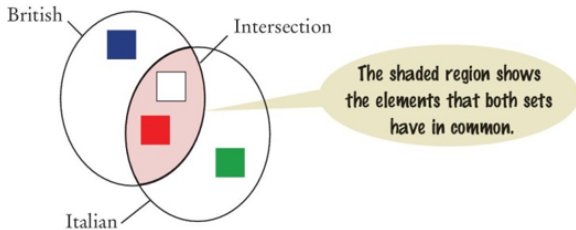
- ▶ In Python we would write:

```
britain.union(italy)
```

and the result is: `{"white", "red", "blue", "green"}`

Intersection

- ▶ The intersection of two sets is the elements which appear in both sets, discarding duplicates.



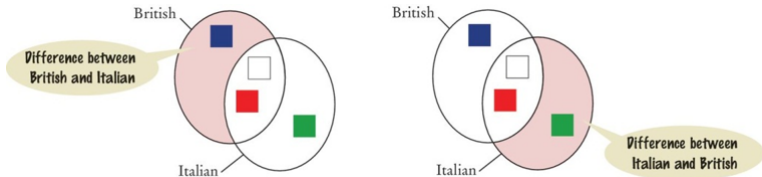
- ▶ In Python we write:

```
britain.intersection(italy)
```

and the result is: {"white", "red"}

Difference

- ▶ The difference between two sets depends on the order of the operands.



- ▶ In python:

```
britain.difference(italy) is: {'blue'}
```

```
italy.difference(britain) is {'green'}
```

Note: the example and diagrams are from Python For Everyone, 2nd Ed., C. Horstmann and R. Necaie

Set operations

A complete list of set operations are **here**

Operation	Description
<code>value in set</code>	returns True if the given value is found in the set
<code>value not in set</code>	returns True if the given value is <i>not</i> found
<code>len(set)</code>	number of elements in the set
<code>str(set)</code>	returns string representation such as <code>"{'a', 'b', 'c'}"</code>
<code>set.add(value)</code>	adds the given value to the set, if not already present
<code>set.clear()</code>	removes all elements
<code>set.isdisjoint(set2)</code>	returns True if there are no elements in common between set and set2
<code>set.pop()</code>	removes and returns one element from the set
<code>set.remove(value)</code>	removes the given value from the set, if present
<code>set.update(sequence)</code>	adds all values from the sequence to the set, if not already present

Dictionaries

Dictionaries

A dictionary keeps an association between a set of **keys** and their **values**. For example: lets take a phonebook, names could be the keys and the numbers could be the values:

POST & TELEGRAPH DEPARTMENT - - - TELEPHONE DIRECTORY			
GOVERNMENT DEPARTMENTS			
Administration -			
13	Secretariat, Central Office	46	Dunwoodie, E. E.
60	Government House, Vailima		
64	Secretary's residence	78	Eden, D. R. (res.) Vailele, 2 rings
115	Assistant Secretary's res.		
Customs Department--		42	Fabricius, P. C., store
23	Collector of Customs	70	Fabricius, P. C., store, Matafele
Education Department--		61	Faleula (Methodist Mission)
50	Director and 16 16 School, 2 rings		
50	Director's residence, 3 rings	92	Gair, C. (res.)
34	Avele School, 3 rings	85	Gibbs, W. H. (res.), 2 rings
34	Headmaster's residence, 2 rings	52	Gold Star Transport (Garage), 2 rings
Harbour Department--		52	Gold Star Transport (res.), 3 rings
12	Harbourmaster	88	Gutz, F. L. A. (res.)
67	Pilot Station and residence	60	Government House
High Court--			
71	Chief Judge, Office	28	Hunt, Dr. (res.)
5	Chief Judge, residence	81	Hoefflich, P.
16	Crown Solicitor and Clerk of Court		
Public Trust Department--			
37	Public Trustee, 2 rings		
Public Works Department--			
45	Engineer in Charge and Office		
Quarantine Station--			
8	Quarantine		
Treasury Department--			
17	Treasury, Cashier, 2 rings		
17	Treasurer, 3 rings		
17	Treasury, Ledgers, 4 rings		
Wireless Station--			
59	Wireless		
New Zealand Repatriation Estates -			
54	General Manager and Office		
88	General Manager, residence		
31	Falelanu Plantation, 3 rings		
31	Tuasimato Plantation, 2 rings		
78	Vailele Plantation, 2 rings		
61	Vaitele Plantation		
78	Vaivase Plantation, 3 rings		
Nelson, O. F. & Co. Ltd.--			
53&9	Manager and Office		
66	Manager's residence		
47	Drapery and Grocery Dept.		
84	Engineering Shop		
79	Newton, H. (res.)		
Papauta School			
52	Paul, E. F. (res.), 3 rings		
Railey, I., storekeeper			
65	Rutherford, D. A. J. (res.), 2 rings		
Samoa Planters, 2 rings			
83	Small, A. F. (res.)		
55	Smythe, A. G. (res.)		
33	Stevadoring Co. Ltd.		
Tattersall, A. J. photographer, 3 rings			
69	Tattersall, A. J. photographer, 3 rings		

By Samoa Post & Telegraph Department, via Wikimedia Commons

Creating and accessing a dictionary

- ▶ There are several ways to create a dictionary. You can start off with an empty dictionary and then add key/value pairs to it:

```
name = {}           # syntax 1 (preferred)
name = dict()       # syntax 2
```

- ▶ You can create a dictionary with initial key/value pairs using the syntax: `name = {key: value, key: value, ..., key: value}`

Note: the colon in between the key and the value.

```
# dictionary with initial data
phonebook = {"Allison": "(520)555-6789", "Marty": "(650)555-1234"}
```

- ▶ Just like sets, you cannot access the elements in a dictionary by position. Neither are the elements stored in the order in which you add them.
- ▶ This means that traversing a dictionary is done in much the same way as a set.

Gotcha - Dictionary keys are unique

- ▶ The keys in a dictionary are unique and if you add a key, value pair where the key already exists, the original key, value pair will be overwritten.
- ▶ You will not receive an error or a warning. Here is an example using the Python shell, you can assume that the dictionary has been set up.

```
>>> # replacing a value in a dictionary
>>> phonebook["Allison"]
'(520)555-6789'
>>> phonebook["Allison"] = "(444)555-8800"
>>> phonebook["Allison"]
'(444)555-8800'
```

Gotcha - More than one key can be associated with the same value

- It is perfectly legal to have two or more keys that refer to the same value. Using the previous example:

```
>>> # dictionary where two keys pair with the same value
>>> phonebook
{'Allison': '(520)555-6789', 'Marty': '(650)555-1234'}
>>> phonebook["Yana"] = "(650)555-1234"    # duplicate value
>>> phonebook["Marty"]
'(650)555-1234'
>>> phonebook["Yana"]
'(650)555-1234'
>>> phonebook
{'Allison': '(520)555-6789', 'Marty': '(650)555-1234',
 'Yana': '(650)555-1234'}
```

Gotcha - Removing dictionary items

- ▶ To remove an item from a dictionary, call the `pop()` method with the key as the argument. Here is an example using the Python shell.

```
>>> phonebook = {}                                # create empty dict
>>> phonebook["Allison"] = "(520)555-6789"        # store a pair
>>> phonebook["Marty"] = "(650)555-1234"          # store another pair
>>> phonebook.pop("Allison")                        # delete entry with key
'(520)555-6789'
>>> phonebook["Allison"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'Allison'
>>>
```

- ▶ You can see that if a key doesn't exist, you will get a `KeyError` exception. We will look at exceptions in a future lecture.

Dictionary operations

Operation	Description
<code>dict[key]</code>	returns the value associated with the given key; raises <code>KeyError</code> if not found
<code>dict[key] = value</code>	sets the value associated with the given key; replaces if already found
<code>del dict[key]</code>	removes the given key and its paired value; raises <code>KeyError</code> if not found
<code>key in dict</code>	returns <code>True</code> if the given key is found
<code>key not in dict</code>	returns <code>True</code> if the given key is <i>not</i> found
<code>len(dict)</code>	number of key/value pairs
<code>str(dict)</code>	returns string representation such as <code>"{'a':1, 'b':2}"</code>
<code>dict.clear()</code>	removes all key/value pairs
<code>dict.get(key, default)</code>	returns the value associated with the given key; returns default if not found
<code>dict.items()</code>	returns the contents of the dictionary as a sequence of (key, value) tuples
<code>dict.keys()</code>	returns the keys in the dictionary as a sequence
<code>dict.pop(key)</code>	returns the value associated with the given key, and removes that key/value pair
<code>dict.update(dict2)</code>	adds all key/value pairs from another dictionary, replacing if keys are already present
<code>dict.values()</code>	returns the values in the dictionary as a sequence

Questions

Question 1

- Consider the following code segment. What is true about these sets?

```
names = set(["Jane", "Joe", "Amy", "Lisa"])
names1 = set(["Joe", "Amy", "Lisa"])
names2 = set(["Jane", "Joe"])
```

- (i) names2 is a subset of names.
- (ii) names2 is not a subset of names.
- (iii) names2 is an intersection of the set names and names1.
- (iv) names2 is a union of names and names1.

Question 2

- Given the following set definitions, which statement correctly determines whether `colours` is a subset of `rainbow`?

```
rainbow = {"red", "orange", "yellow", "green", "blue", "indigo", "violet"}
colours = {"red", "orange", "blue", "green"}
```

- (i)

```
if colours in rainbow :  
    print("rainbow colours")
```
- (ii)

```
if colours.issubset(rainbow) :  
    print("rainbow colours")
```
- (iii)

```
if rainbow.issubset(colours) :  
    print("rainbow colours")
```
- (iv)

```
if colours is rainbow :  
    print("rainbow colours")
```

Question 3

- Given the following dictionary definition

```
favoriteFoods = {"Peg": "burgers", "Cy": "hotdogs", "Bob": "apple p
```

Which code segment correctly prints the dictionary, both the key and value, in alphabetical order by the person's name?

(i) `print(favoriteFoods)`

(ii) `for name in sorted(favoriteFoods) :`
`print(name, favoriteFoods[name])`

(iii) `for name in (favoriteFoods) :`
`print(name, favoriteFoods[name])`

(iv) `for name in sorted(favoriteFoods) :`
`print(favoriteFoods[name])`

Question 4

- Given the dictionary `periodicTable` that contains the periodic table of the elements, which of the following correctly prints the values stored in the table, one per line?

(i) `print(periodicTable)`

(ii) `print(values(periodTable))`

(iii) `for value in periodicTable :
 print(value)`

(iv) `for value in periodicTable.values() :
 print(value)`

Answer 1

- Consider the following code segment. What is true about these sets?

```
names = set(["Jane", "Joe", "Amy", "Lisa"])
names1 = set(["Joe", "Amy", "Lisa"])
names2 = set(["Jane", "Joe"])
```

- (i) names2 is a subset of names. **True**
- (ii) names2 is not a subset of names. **False**
- (iii) names2 is an intersection of the set names and names1. **False**
- (iv) names2 is a union of names and names1. **False**

Answer 2

- ▶ Given the following set definitions, which statement correctly determines whether colours is a subset of rainbow?

```
rainbow = {"red", "orange", "yellow", "green", "blue", "indigo", "violet"}  
colours = {"red", "orange", "blue", "green"}
```

- (ii)

```
if colours.issubset(rainbow) :  
    print("rainbow colours")
```

Answer 3

```
(ii) for name in sorted(favoriteFoods) :  
        print(name, favoriteFoods[name])
```

Answer 4

```
(iv) for value in periodicTable.values() :  
      print(value)
```